SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing techniques. (BL4,BL5)
Assessment Tool Weightage: 40%

Total Marks:40

QUESTIONS: 05

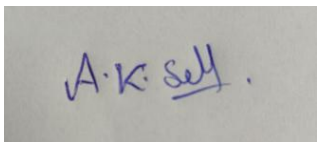
Annexure A

MOCK INTERVIEW

Code of Conduct:

I A K Selva Prabhu(192421224) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Mock Interview – Sample Questions (5 Questions)

Mock Interview Test Format for Object Oriented Analysis and Design (OOAD)

Section A	**Self Introduction & Fundamentals**	**3 Minutes**	**5**
Section B	**Technical Questions (OOAD Concepts)**	**7 Minutes**	**10**
Section C	**UML Modeling & Design**	**7 Minutes**	**10**
Section D	**Case Study / Problem Solving**	**8 Minutes**	**10**
Section E	**HR & Career-Oriented Questions**	**5 Minutes**	**5**

SECTION A – SELF INTRODUCTION (5 Marks)

Evaluation Criteria

- Communication Skills
- Confidence
- Technical Awareness
- Academic Background

Sample Questions

1. Tell me about yourself.
2. Why did you choose Computer Science?
3. Explain your favorite subject in OOAD.
4. What software development tools have you used?
5. Describe a project where you applied object-oriented concepts.

SECTION B – TECHNICAL QUESTIONS (15 Marks)

Basic Level

Q1. What is Object-Oriented Analysis and Design?

Q2. Differentiate between Analysis and Design.

Q3. Explain Encapsulation with an example.

Q4. What is Inheritance? Mention its types.

Q5. Define Polymorphism and its benefits.

Q6. What is Abstraction?

Q7. Difference between Aggregation and Composition.

Q8. What is Coupling and Cohesion?

Q9. What is a Class and an Object?

Q10. Explain Object Responsibility.

SECTION C – UML MODELING (15 Marks)

Q11. What is UML?

Q12. When would you use a Use Case Diagram?

Q13. Draw and explain a Class Diagram for a Library Management System.

Q14. Differentiate Sequence Diagram and Activity Diagram.

Q15. Explain State Diagram with an example.

Q16. What is a Deployment Diagram?

Q17. How do Component Diagrams help software architects?

Q18. What is Multiplicity in UML?

SECTION D – CASE STUDY / PROBLEM SOLVING (10 Marks)

Case Study 1

A hospital wants a system to manage:

- Patients
- Doctors
- Appointments
- Billing

Questions

1. Identify the classes.
2. Draw a simple class diagram.
3. Suggest relationships among classes.
4. Which UML diagrams would you use for analysis?
5. Explain how inheritance can be applied.

Case Study 2

An Online Shopping System supports:

- Customers

- Products
- Cart
- Orders
- Payments

Questions

1. Identify actors and use cases.
2. Design a use case diagram.
3. Suggest suitable design patterns.
4. Explain how polymorphism can be used in payment processing.

SECTION E – HR & PROFESSIONAL QUESTIONS (5 Marks)

Q19. Why should we hire you?

Q20. What are your strengths and weaknesses?

Q21. How do you handle project deadlines?

Q22. Have you worked in a team project? Explain your role.

Q23. Where do you see yourself in five years?

RUBRICS FOR CALCULATION:

MOCK INTERVIEW

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)

Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

SECTION B – TECHNICAL QUESTIONS

Q1. What is Object-Oriented Analysis and Design?

Object-Oriented Analysis and Design (OOAD) is a software development approach that models a system using **objects and classes**.

- **Analysis** identifies system requirements, objects, classes, and relationships.
- **Design** defines how these objects will work together.
- It uses concepts such as **encapsulation, inheritance, abstraction, and polymorphism**.
- UML diagrams are commonly used in OOAD.

Q2. Differentiate between Analysis and Design.

Analysis	**Design**
Focuses on **what the system should do**	Focuses on **how the system will do it**
Identifies requirements	Defines system architecture
Identifies classes and objects	Defines class structure and methods
Problem-oriented	Solution-oriented
Done before detailed design	Follows analysis

Q3. Explain Encapsulation with an example.

Encapsulation means combining data and methods into a single class and restricting direct access to the data.

Example:

```
Class BankAccount

   private balance
```

deposit(amount)

withdraw(amount) getBalance()

End Class

Q4. What is Inheritance? Mention its types.

Inheritance is an OOP concept where one class acquires the properties and methods of another class.

Example:

Vehicle

|

↓

Car

Car inherits properties from Vehicle.

Types:

1. Single inheritance
2. Multilevel inheritance
3. Hierarchical inheritance
4. Multiple inheritance
5. Hybrid inheritance

Q5. Define Polymorphism and its benefits.

Polymorphism means **one interface or method can have different implementations**.

Example:

Animal

↓

Dog → Sound = Bark Cat

→ Sound = Meow

Benefits:

- Improves flexibility
- Supports code reuse
- Reduces code duplication
- Makes systems easier to extend
- Supports maintainability

Q6. What is Abstraction?

Abstraction means hiding unnecessary implementation details and showing only the important features to the user.

Example:
When using an ATM, the user selects **Withdraw Money**, but does not need to know the internal banking process.

Q7. Difference between Aggregation and Composition.

Aggregation	Composition
Weak relationship	Strong relationship
Child can exist independently	Child depends on parent
Represented by empty diamond	Represented by filled diamond
Example: Department–Teacher	Example: House–Room

Q8. What is Coupling and Cohesion?

Coupling: Degree of dependency between different modules.

- **Low coupling** is preferred.
- It makes the system easier to modify.

Cohesion: Degree to which elements within a module belong together.

- **High cohesion** is preferred.
- It makes modules focused and easier to maintain. **Ideal design: Low**

Coupling + High Cohesion Q9. What is a Class and an Object?

Class: A blueprint or template that defines properties and methods.

Object: An actual instance of a class.

Example:

Class: Student

Objects:

Student1

Student2

Student3

The class defines the structure, while objects represent real entities.

Q10. Explain Object Responsibility.

Object responsibility refers to the tasks or actions that an object is responsible for performing.

Example:

Class: BankAccount

Responsibilities:

- Deposit money

- Withdraw money

- Check balance

- Generate transaction details

Assigning responsibilities properly improves **cohesion, maintainability, and system organization**.

SECTION C – UML MODELING Q11.

What is UML?

UML (Unified Modeling Language) is a standard visual language used to **visualize, specify, design, and document software systems**.

Common UML diagrams include:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram
- State Diagram
- Component Diagram
- Deployment Diagram

Q12. When would you use a Use Case Diagram?

A **Use Case Diagram** is used to show the interaction between **users/actors and a system**.

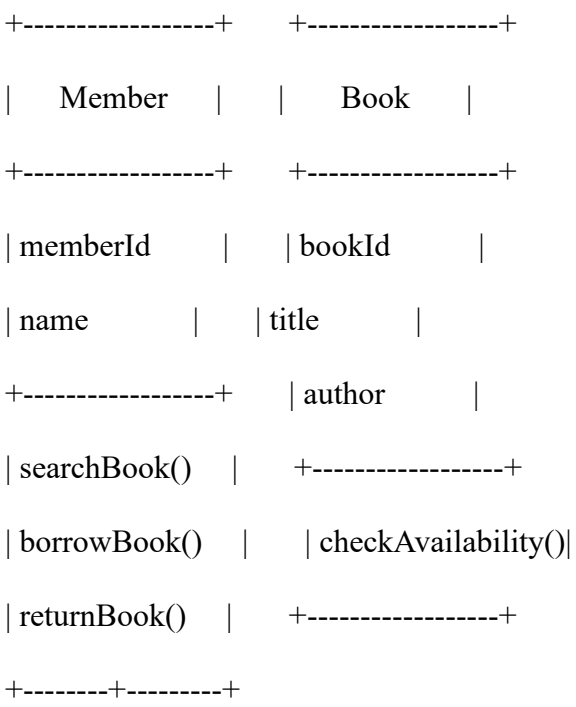
It is useful during:

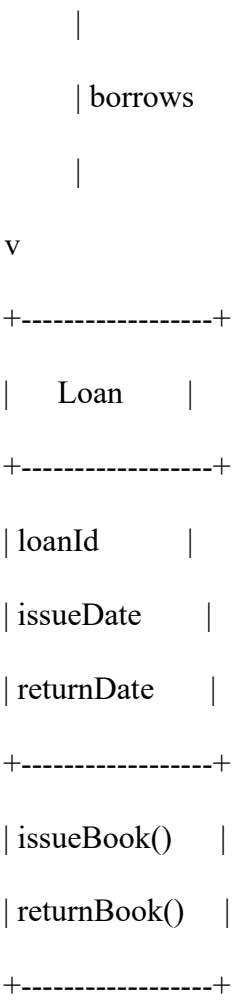
- Requirement analysis
- Identifying system functions
- Identifying actors
- Understanding user-system interaction
- Communicating requirements with customers

Example: In an online shopping system, a customer can **search products, add items to cart, place orders, and make payments**.

Q13. Draw and explain a Class Diagram for a Library Management System.

Simple Class Diagram





Explanation:

- Member represents library users.
- Book stores book information.
- Loan manages borrowing and returning.
- A member can borrow books through a loan transaction.

Q14. Differentiate Sequence Diagram and Activity Diagram.

Sequence Diagram	**Activity Diagram**
Shows interaction between objects	Shows workflow of activities
Focuses on time/order	Focuses on process flow
Uses lifelines and messages	Uses activities and decisions
Useful for object interaction	Useful for business processes
Example: Login process	Example: Online shopping workflow

Q15. Explain State Diagram with an example.

A **State Diagram** shows the different states of an object and how it changes from one state to another due to events.

Example: Book

Available

|

| Borrow

↓

Borrowed

|

| Return

↓

Available

Another state could be:

Available → Reserved → Borrowed → Returned It is

useful for systems where objects have different states.

Q16. What is a Deployment Diagram?

A **Deployment Diagram** shows the physical arrangement of hardware and software components in a system.

Example:

[User Computer]

|

↓

[Web Server]

|

↓

[Application Server]

|

↓

[Database Server]

It helps developers understand **where software components are deployed**.

Q17. How do Component Diagrams help software architects?

Component diagrams show the major **software components and their dependencies**.

They help architects to:

- Understand system structure
- Identify component dependencies
- Plan modular architecture
- Improve maintainability
- Support system integration
- Identify reusable components **Q18. What is Multiplicity in UML?**

Multiplicity specifies how many objects of one class can be associated with another class.

Common values:

- 1 → Exactly one
- 0..1 → Zero or one
- 1..* → One or more
- 0..* → Zero or more

Example:

Doctor 1 -------- 0..* Appointment

This means **one doctor can have zero or many appointments**.

SECTION D – CASE STUDY / PROBLEM SOLVING

Case Study 1 – Hospital Management System

1. Identify the classes.

Main classes:

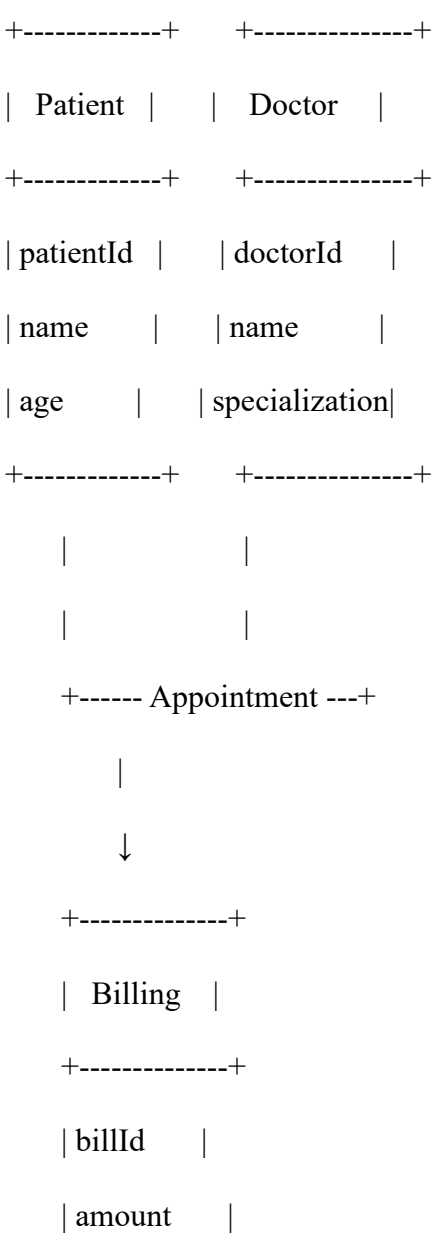
- Patient
- Doctor

- Appointment
- Billing

Additional classes can include:

- Hospital
- MedicalRecord
- Payment

2. Simple Class Diagram



| status |

+--------------+

3. Relationships among classes

- Patient **books** Appointment.
- Doctor **attends** Appointment.
- Appointment **generates** Billing.
- Patient **has** Billing.
- Doctor can have **many appointments**. • Patient can have **many appointments**.

4. Which UML diagrams would you use for analysis?

Suitable UML diagrams are:

1. **Use Case Diagram** – identifies users and system functions.
2. **Class Diagram** – identifies classes and relationships.
3. **Activity Diagram** – represents hospital workflows.
4. **Sequence Diagram** – shows interactions during appointment booking.
5. **State Diagram** – represents appointment states.

5. Explain how inheritance can be applied.

Inheritance can be used with a common Person class.

Person

/ \

/ \

Patient Doctor

Patient and Doctor inherit common properties such as:

- Name
- Age
- Phone number
- Address

This reduces duplication and improves code reuse.

Case Study 2 – Online Shopping System

1. Identify actors and use cases.

Actor:

- Customer
- Payment Service/Gateway
- Admin

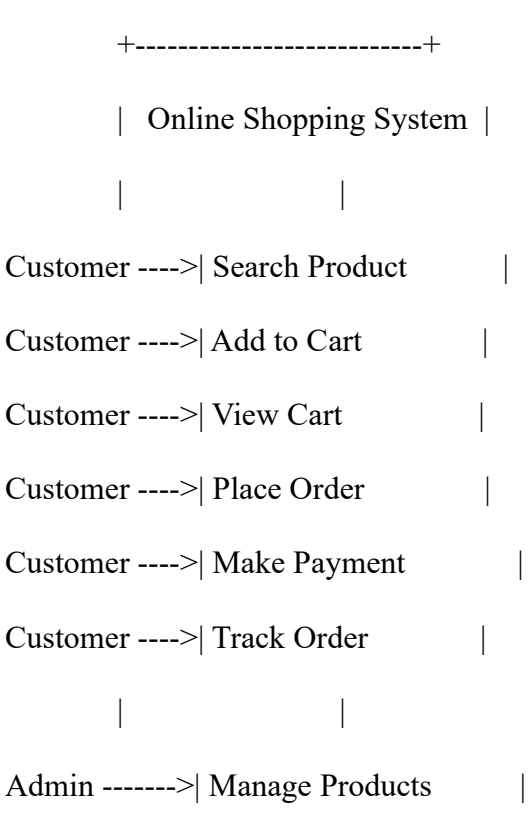
Customer Use Cases:

- Register/Login
- Search Products
- Add to Cart

- View Cart
- Place Order
- Make Payment
- Track Order **Admin Use Cases:**

- Add Product
- Update Product
- Remove Product
- Manage Orders

2. Simple Use Case Diagram



Admin ------->| Manage Orders |

+--------------------------+

|

↓

Payment Gateway

3. Suggest suitable design patterns.

Suitable patterns include:

1. **Factory Pattern** – create different payment types.
2. **Singleton Pattern** – manage a shared service such as configuration.
3. **Observer Pattern** – send order/status notifications.
4. **Strategy Pattern** – support different payment or shipping methods.
5. **MVC Pattern** – separate user interface, business logic, and data.

4. Explain how polymorphism can be used in payment processing.

A common interface can be created:

Payment

|

+---- CreditCardPayment

+---- UPIPayment +----

NetBankingPayment

Each class implements:

makePayment()

But each implementation works differently.

CreditCard → makePayment()

UPI → makePayment()

NetBanking → makePayment()

Thus, the same method can perform different payment operations. This is **polymorphism**.

SECTION E – HR & PROFESSIONAL QUESTIONS

Q19. Why should we hire you?

Answer:

I am a motivated and quick-learning student with a strong interest in technology and software development. I have knowledge of programming, databases, AI and web technologies. I am willing to learn new skills, work effectively in a team, and take responsibility for my tasks. I believe my positive attitude and technical interest will allow me to contribute to the organization.

Q20. What are your strengths and weaknesses?

Answer:

Strengths:

- Quick learner
- Problem-solving ability
- Teamwork
- Time management
- Positive attitude

Weakness:
Sometimes I spend extra time making sure my work is correct. I am improving this by setting priorities and managing my time more effectively.

Q21. How do you handle project deadlines?

Answer:

I first divide the project into smaller tasks and prioritize them based on importance. I create a schedule and track my progress regularly. If I face a problem, I try to solve it early or discuss it with my team. This helps me complete the project on time without compromising quality.

Q22. Have you worked in a team project? Explain your role.

Answer:

Yes, I have worked on team-based academic projects. My role involved contributing to the **design, development, testing, and documentation** of the project. I coordinated with team members, shared ideas, completed assigned tasks, and helped solve technical problems. This experience improved my communication and teamwork skills.

Q23. Where do you see yourself in five years?

Answer:

In five years, I see myself as a skilled technology professional with strong expertise in software development and emerging technologies such as **AI and data science**. I want to work on real-world projects, take greater responsibilities, continuously improve my skills, and contribute to the growth of the organization.